FarmaPoint
Laboratorio 09

Introducción a React, JSX y Vite

Interfaz de inventario, categorías y ventas con datos locales.

Integrantes	Curso / entrega
Dario Maza Sheila Aguirre Jose Contreras	Desarrollo de Aplicaciones Empresariales 4 - C24 - Sección CD 17 de octubre de 2026

Docente: Yunior Bestard Aroche

Repositorio: https://github.com/SAguirreT/django_project

Commit del frontend: 6a3a66512b4a56ed9eda498ce7af54b3e3598742

Entrega en PDF y con los integrantes indicados. Se desarrollan los 14 ejercicios de la guía, con justificación, código y resultados verificables. Los datos de productos y clientes son ficticios.

Alcance: React complementa al proyecto Django. Los datos se escriben en arreglos; en esta sesión no se consume la API REST ni se modifican registros de la base de datos.

Las imágenes de la aplicación son capturas reales de Edge. Las imágenes de código y comandos muestran un visor de archivos y registros de ejecución, identificado como tal; no son fotografías de una terminal.

1. Verificar Node.js y npm

Se verificó Node.js **v24.15.0** y npm **11.12.1**. Node permite ejecutar las herramientas de desarrollo de JavaScript; npm instala los paquetes. React es la biblioteca que construye la interfaz y Vite inicia el servidor y genera la compilación.

Elemento	Python / Django	JavaScript / React
Lenguaje	Python; HTML en templates	JavaScript y JSX
Gestor de paquetes	pip	npm
Dependencias declaradas	requirements.txt	package.json y package-lock.json
Dependencias instaladas	Entorno virtual: venv/Lib/site-packages	node_modules/
Servidor de desarrollo	python manage.py runserver	npm run dev (ejecuta Vite)

Justificación: Node.js no equivale a Django: es un entorno de ejecución. React se encarga de la vista; Django mantiene modelos, validaciones, persistencia y API.



E1-E2. Versiones y salida registrada de instalación y arranque.

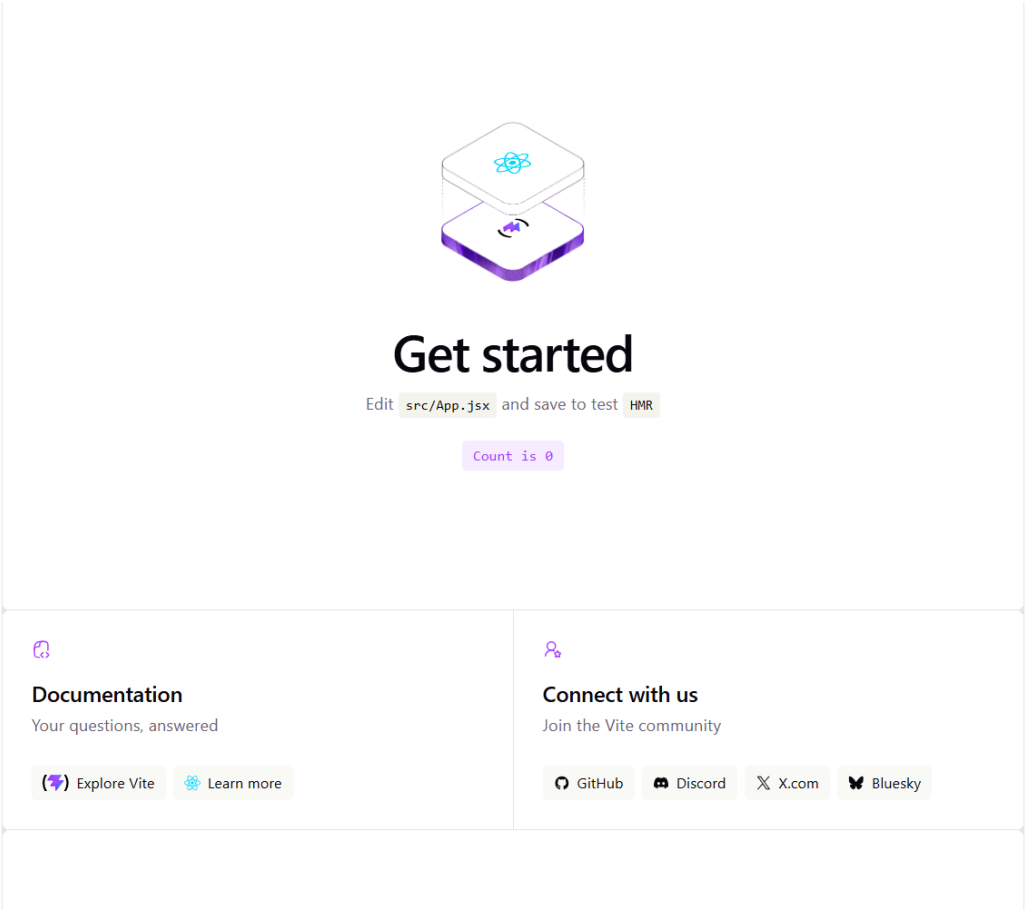
2. Crear y ejecutar el proyecto

Se creó **frontend/** en la raíz del mismo repositorio. Se usó la plantilla oficial React + JavaScript de create-vite, se ejecutó npm install y se inició Vite en el puerto 5173. La captura siguiente conserva la página inicial antes de sustituirla por FarmaPoint.

Secuencia: npm create vite@latest frontend -- --template react; cd frontend; npm install; npm run dev. La URL solicitada es http://localhost:5173/. Las pruebas finales usan http://127.0.0.1:5173/, que apunta al servidor IPv4 local.

Incidencia resuelta: el lanzador inicial devolvió EBADF al crear archivos. Se ejecutó directamente el CLI oficial create-vite 9.2.1 descargado por npm con --template react --no-interactive. La instalación terminó correctamente. Se conservan los registros de error y recuperación; no se presenta el intento fallido como una ejecución exitosa.

Para los scripts npm de este entorno Windows se utilizó PowerShell como script-shell. Se configuró Vite con --configLoader native para evitar el problema de archivos temporales. El README documenta el ajuste local.



E2. Página inicial real de la plantilla, antes de editar App.jsx.

3. Reconocer la estructura

Archivo / carpeta	Función y comparación con Django
index.html	Contenedor HTML único con div#root. Se parece al esqueleto de base.html, pero Vite lo sirve sin procesar templates de Django.
src/main.jsx	Punto de entrada: createRoot monta App dentro de root. Es un arranque de interfaz; no reemplaza manage.py.
src/App.jsx	Componente principal: datos locales, expresiones y JSX. Cumple parte del papel visual que tenía el template.
package.json	Declara paquetes y scripts. Equivale parcialmente a requirements.txt; además contiene comandos de desarrollo.
package-lock.json	Fija las versiones resueltas para repetir la instalación.
node_modules/	Paquetes descargados por npm; equivalente práctico a los paquetes del entorno virtual. No se versiona.



E3. Inventario de archivos del frontend; dependencias y compilación se conservan solo localmente.

4. Primer JSX con expresiones

Se vaciaron App.css e index.css para observar el HTML sin estilos de la plantilla. El primer componente mostró el nombre de la aplicación en h1. **{nombre}** inserta una variable y **{2 + 3}** evalúa un cálculo. Esta etapa se capturó antes de construir la interfaz final.

Justificación: las llaves permiten evaluar expresiones de JavaScript dentro del JSX. No son las dobles llaves del motor de templates de Django: se ejecutan en el entorno de la interfaz React.

Primer JSX / App.jsx

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
import './App.css'
const nombre = 'FarmaPoint'
export default function App() { return <main><h1>{nombre}</h1><p>Productos de
ejemplo: {2 + 3}</p></main> }
```

E4. Código de la etapa inicial.



E4. Resultado: FarmaPoint y cinco productos de ejemplo (recorte de la captura inicial).

5. Lista con map() y key

El arreglo productos contiene cinco registros. Cada objeto conserva los campos de ProductoSerializer: id, nombre, descripcion, precio, stock, categoria y categoria_nombre. El precio sigue siendo una cadena, como en la respuesta JSON del Lab 08.

La lista se construye con **visibles.map(p => ...)**; visibles se obtiene filtrando productos. Cada artículo usa **key={p.id}**. La función moneda utiliza **Number(valor).toFixed(2)** para mostrar dos decimales.

Equivalencia: map transforma cada objeto en un elemento JSX. En Django se recorría la colección con {% for producto in productos %} ... {% endfor %}. La key permite que React identifique cada elemento cuando cambia la lista; no se usa la posición del arreglo.

App.jsx / expresiones, filtros y cálculo

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
161  const moneda = valor => `S/ ${Number(valor).toFixed(2)}`

163  const totalVenta = venta => venta.detalles.reduce((s, d) => s + d.cantidad
* Math.round(Number(d.precio_unitario) * 100), 0) / 100

166  const nombre = 'FarmaPoint'

167  const [busqueda, setBusqueda] = useState('')

168  const [categoria, setCategoria] = useState(null)

169  const visibles = productos.filter(p =>
p.nombre.toLowerCase().includes(busqueda.toLowerCase()) && (categoria === null
|| p.categoria === categoria))

170  const unidades = productos.reduce((s, p) => s + p.stock, 0)
```

E5. Conversión de importes, datos calculados y filtro local.

App.jsx / map, key y condiciones

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
178  {visibles.length === 0 ? <p className="empty">No hay registros</p> :
visibles.map(p => <article className="product" key={p.id} data-
testid="producto"><div><small>{p.categoria_nombre}</small><h3>{p.nombre}</h3><p>
{p.descripcion}</p></div><div className="price"><strong>{moneda(p.precio)}
</strong><span className={p.stock > 0 ? 'badge available' : 'badge sold'}>
{p.stock > 0 ? `Disponible · ${p.stock}` : 'Agotado'}</span>

180  {p.stock > 0 && p.stock <= 3 && <small className="warning">¡Últimas
unidades!</small>}
```

E5-E6. Lista con key basada en id y mensajes condicionales.

5. Arreglo de productos en App.jsx

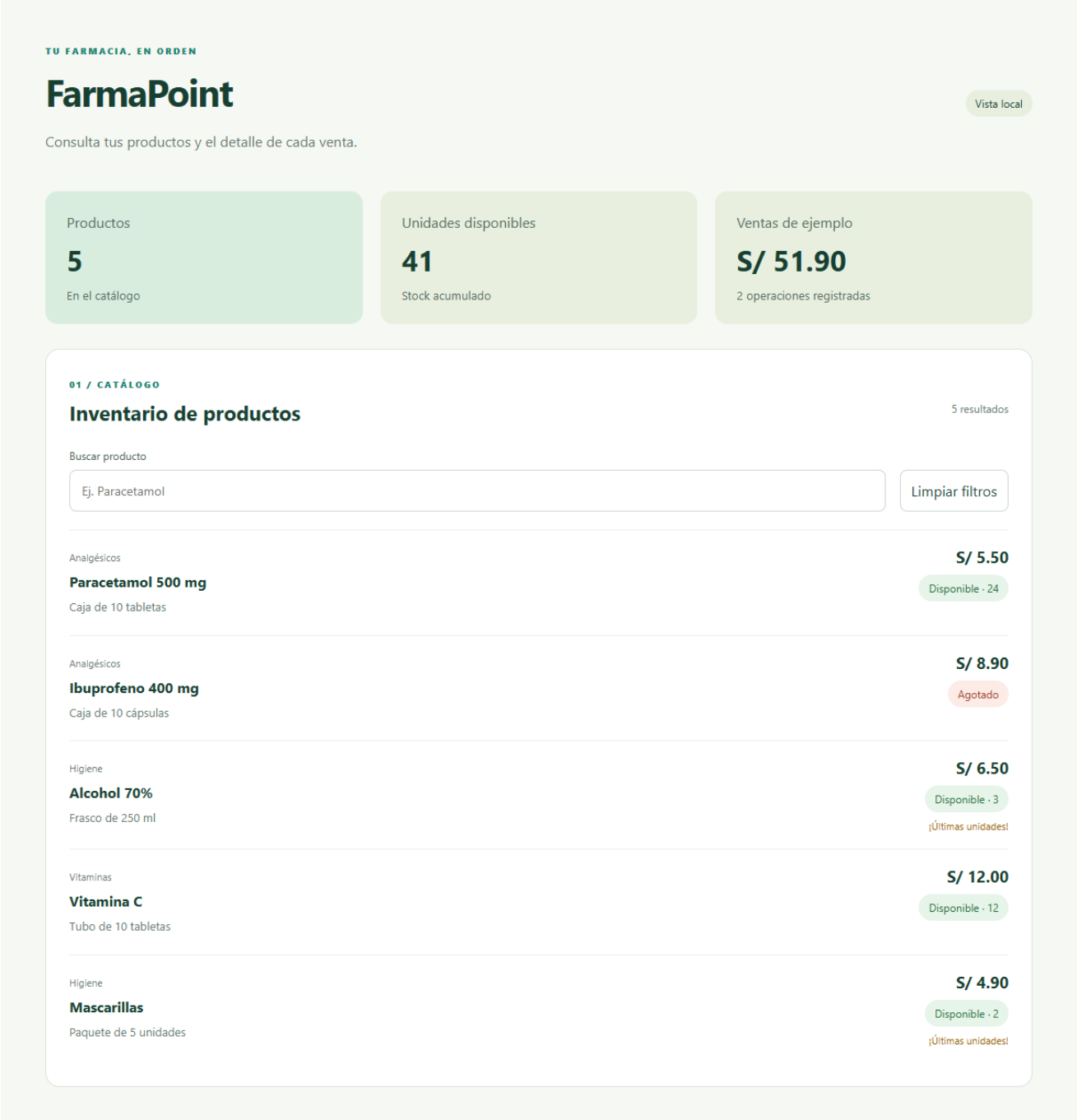
App.jsx / cinco productos con forma de ProductoSerializer

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
004  // Datos locales con los campos de los serializers del Lab 08.
005  const productos = [
006    {
007      "id": 1,
008      "nombre": "Paracetamol 500 mg",
009      "descripcion": "Caja de 10 tabletas",
010      "precio": "5.50",
011      "stock": 24,
012      "categoria": 1,
013      "categoria_nombre": "Analgésicos"
014    },
015    {
016      "id": 2,
017      "nombre": "Ibuprofeno 400 mg",
018      "descripcion": "Caja de 10 cápsulas",
019      "precio": "8.90",
020      "stock": 0,
021      "categoria": 1,
022      "categoria_nombre": "Analgésicos"
023    },
024    {
025      "id": 3,
026      "nombre": "Alcohol 70%",
027      "descripcion": "Frasco de 250 ml",
028      "precio": "6.50",
029      "stock": 3,
030      "categoria": 2,
031      "categoria_nombre": "Higiene"
032    },
033    {
034      "id": 4,
035      "nombre": "Vitamina C",
036      "descripcion": "Tubo de 10 tabletas",
037      "precio": "12.00",
038      "stock": 12,
039      "categoria": 3,
040      "categoria_nombre": "Vitaminas"
041    },
042    {
043      "id": 5,
044      "nombre": "Mascarillas",
045      "descripcion": "Paquete de 5 unidades",
046      "precio": "4.90",
047      "stock": 2,
048      "categoria": 2,
049      "categoria_nombre": "Higiene"
050    }
051  ]
```

E5. Cinco objetos con id y decimales como texto; campos del serializer de productos.

5. Evidencia de los cinco productos



E5. Catálogo completo: cinco productos, 41 unidades y precios con dos decimales.

6. Renderizado condicional

El ternario comprueba p.stock > 0: muestra **Disponible** con la cantidad o **Agotado**. El operador && añade **¡Últimas unidades!** cuando el stock está entre 1 y 3. Ibuprofeno tiene cero unidades; Alcohol tiene tres y Mascarillas tiene dos.

Otro ternario muestra **No hay registros** cuando visibles.length === 0. Se comprobó tanto una búsqueda sin coincidencias como la sustitución temporal del arreglo productos por []. Después de la prueba se restauraron los cinco registros.

Justificación: el ternario selecciona una de dos salidas. && incorpora contenido solo cuando la condición es verdadera. La condición usa comparaciones booleanas para evitar que React muestre accidentalmente el número 0.



E6. Resultado real con productos = []; el total de productos y stock baja a cero.

7. Relación 1:N con listas anidadas

ventas contiene dos objetos con cliente y detalles. Cada detalle incluye un producto, cantidad, precio_unitario y subtotal. Esta forma corresponde a VentaReadSerializer y DetalleVentaReadSerializer. Se aplica un map por venta y otro por detalle, con key={v.id} y key={d.id}.

Total calculado: la venta 101 suma 2 × 5.50 + 1 × 12.00 + 1 × 4.90 = **S/ 27.90**. La venta 102 suma 2 × 12.00 = **S/ 24.00**. El total no se copia del campo total: se recalcula con reduce. Para la suma se convierten los importes a céntimos enteros y se divide entre 100 al final.

Justificación: Venta tiene muchos DetalleVenta. DetalleVenta es también el modelo intermedio entre Venta y Producto, y conserva datos propios de la relación: cantidad y precio unitario.

App.jsx / relación anidada y estilo en línea

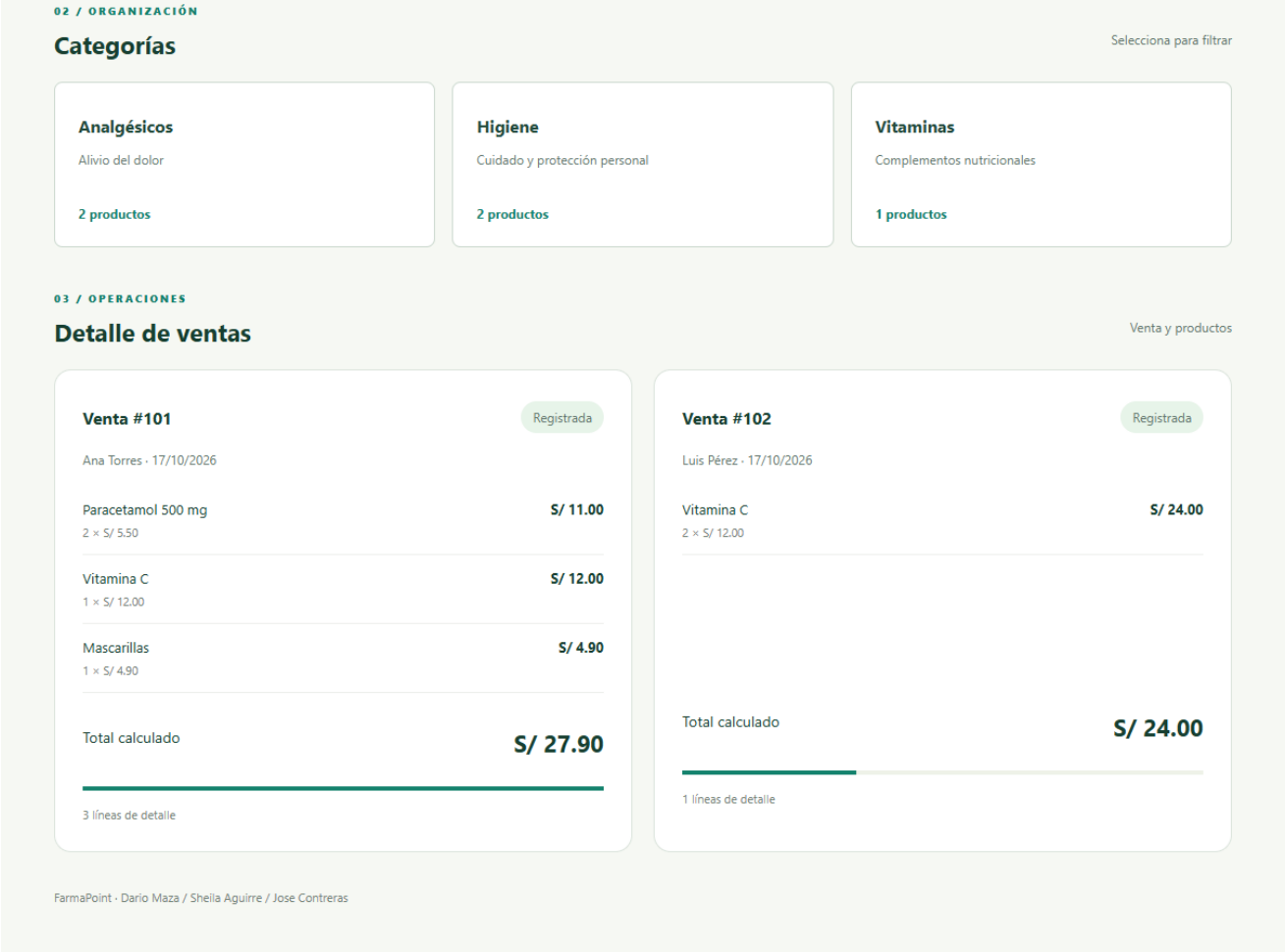
FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
186  {ventas.map(v => <article className="panel sale" key={v.id} data-
testid="venta"><div className="section-head"><h3>Venta #{v.id}</h3><span
className="badge available">Registrada</span></div><p>{v.cliente.nombre} · {new
Date(v.fecha).toLocaleDateString('es-PE')}</p><ul>{v.detalles.map(d => <li key=
{d.id}><span>{d.producto.nombre}<small>{d.cantidad} ×
{moneda(d.precio_unitario)}</small></span><strong>{moneda(d.cantidad *
Number(d.precio_unitario))}</strong></li>)}</ul><div className="total">
<span>Total calculado</span><strong data-testid="total-venta">
{moneda(totalVenta(v))}</strong></div>

188  <div className="track"><div style={{ width: `${Math.min(v.detalles.length /
3 * 100, 100)}%`, backgroundColor: '#14826d', height: '4px' }} /></div><small>
{v.detalles.length} líneas de detalle</small>
```

E7. map anidado, key en ambos niveles, total y conteo.

7. Evidencia de categorías y ventas



E7-E12. Tres categorías y dos ventas. Totales y conteos calculados: tres líneas y una línea.

8. Estilos y flujo completo

Se aplican clases mediante className y reglas propias en App.css: .app define el contenedor, aside el menú, .available el estado disponible y .sold el agotado, entre otras. Las media queries adaptan el diseño al ancho de un teléfono.

La barra de cada venta usa **style={{ width: ..., backgroundColor: ..., height: ... }}**. Es un objeto JavaScript; backgroundColor se escribe en camelCase. Su ancho depende de la cantidad de líneas de detalle.

Se cambió temporalmente Paracetamol 500 mg por Paracetamol actualizado en el arreglo y Vite actualizó la página mediante HMR. Luego se restauró el dato. npm run lint y npm run build terminaron con código 0; sus registros aparecen en el ejercicio 13.

App.css / estilos propios (extractos)

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

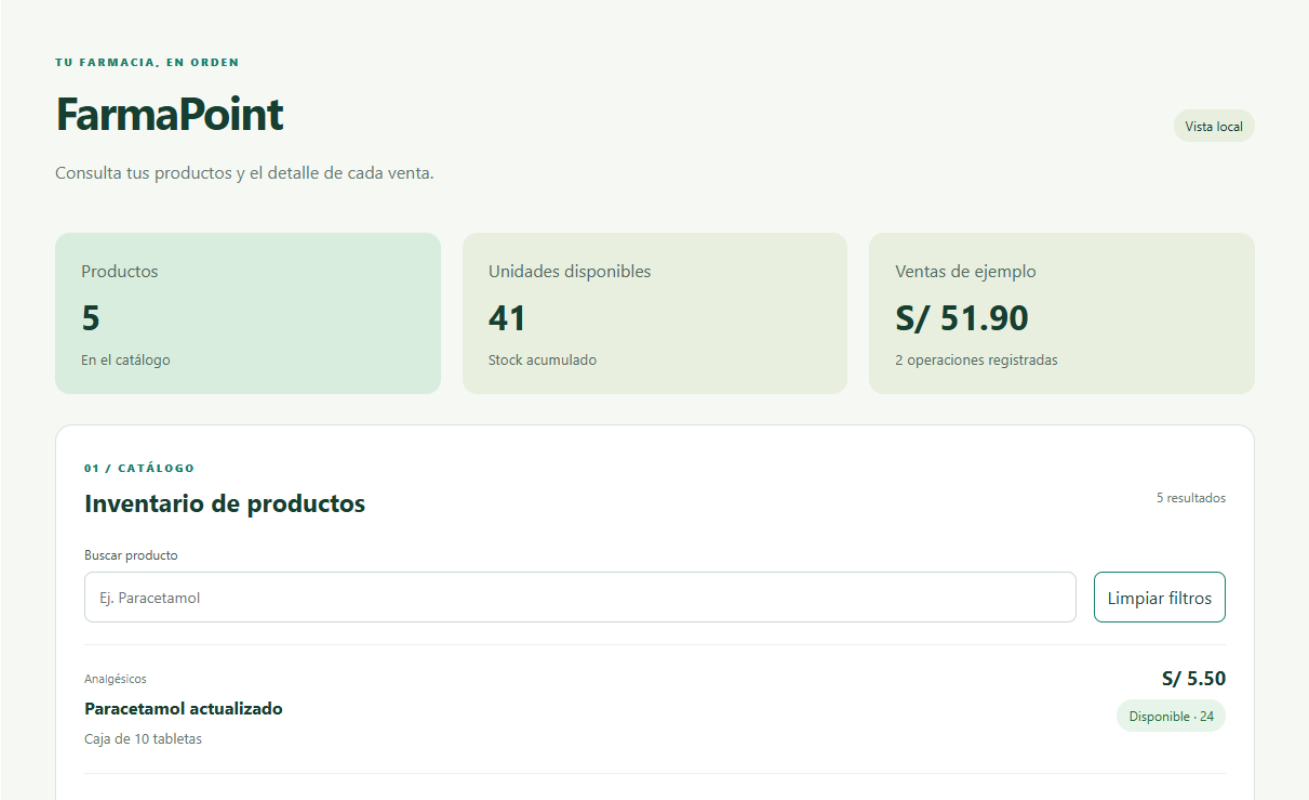
```
/* Estilos propios: estructura, estados de stock y diseño adaptable. */
.app {
  display:flex;
  min-height:100vh
}
aside {
  width:230px;
  background:#113d36;
  color:white;
  padding:34px 24px;
  position:fixed;
  height:100vh
}
.available {
  background:#e7f4e9;
  color:#227140
}
.sold {
  background:#fdebe6;
  color:#a34027
}
.product .warning {
  color:#9a6100
}
```

E8. Extractos reales de App.css: estructura y colores de estado.

8. Diferencia con Django Templates

En la Semana 6, Django recibía la solicitud, consultaba los datos y renderizaba HTML en el servidor mediante templates, extends, include, for e if. Aquí React monta componentes en el navegador y recalcula la vista cuando cambia el estado. La búsqueda y las categorías usan useState para actualizar la lista sin solicitar otra página.

React no reemplaza Django. Django mantiene la base de datos y la API. Para conectarlos falta sustituir los arreglos por datos obtenidos con fetch u otro cliente HTTP, administrar carga y errores, adaptar la paginación results si está habilitada y configurar la URL de API. Si frontend y backend usan orígenes distintos, se necesita un proxy de desarrollo o CORS; las operaciones autenticadas también requieren manejar sus credenciales y permisos. Esa conexión no se implementa en el Lab 09.



E8. Cambio temporal de un dato visible sin recargar manualmente la página.

9. Identificar la investigación propia

La investigación de FarmaPoint y la aplicación de prácticas están en el mismo repositorio. Para la Parte 2 se reutiliza la interfaz de farmacia, identificando de forma explícita sus siete entidades: cinco originales, una de relación 1:1 y el modelo intermedio M:N.

Grupo	Entidades y relación
Cinco originales	Categoria, Producto, Proveedor, Cliente y Venta.
Entidad 1:1	PerfilCliente: OneToOneField hacia Cliente.
Modelo intermedio	DetalleVenta: une Venta y Producto con cantidad y precio_unitario.

Criterio	Aplicación concreta
Dos entidades mostradas	Producto y Categoria; ambas con arreglo propio y map con id.
Campo condicional	Producto.stock: agotado, disponible y últimas unidades.
Relación anidada	Venta -> detalles -> producto; DetalleVenta es el intermedio M:N.
JSON de referencia	GET /api/v1/productos/, GET /api/v1/categorias/ y GET /api/v1/ventas/.
Forma del arreglo	Se modelan los objetos de cada recurso; si la respuesta está paginada, corresponden a los elementos de results, no al envoltorio count/next/previous.

Justificación: no hace falta mostrar las siete entidades en pantalla; la guía pide escoger dos y una relación. Proveedor y PerfilCliente se identifican en models.py, pero no se inventa un endpoint para ellos. Los campos usados se contrastaron con serializers.py del Lab 08.

9. Checklist de la Parte 2

Criterio	Verificación
1. Node y npm	v24.15.0 y 11.12.1; ejercicio 1.
2. frontend y servidor	Carpeta raíz y Vite en el puerto 5173; ejercicios 2-3.
3. JSX y expresiones	Título {nombre}, {productos.length} y {unidades}; ejercicios 4-5.
4. Dos listas con key	Producto y Categoria usan sus id; ejercicios 5 y 10.
5. Condicional y lista vacía	Ternario, && y productos = []; ejercicios 6 y 11.
6. Relación anidada	ventas.map y detalles.map; ejercicios 7 y 12.
7. className y CSS	App.css e inline style; ejercicios 8 y 11.
8. Consola, lint y build	Sin eventos de error/advertencia; comandos con salida 0; ejercicio 13.
9. Comparación y API	Diferencias con templates y pasos futuros explicados en ejercicio 8.

El checklist se basa en el estado final probado. La prueba automatizada también verificó cinco productos, tres categorías, dos ventas, filtros, importes, HMR, ausencia de desbordamiento horizontal a 390 px y cero solicitudes a /api/.

10. JSX y listas de la investigación

Los arreglos productos y categorias se escribieron dentro de App.jsx. Producto conserva categoria como identificador y categoria_nombre como campo de lectura. Categoria contiene id, nombre y descripcion. No se sustituyó esta forma por campos inventados para la pantalla.

La lista de categorías usa categorias.map(c => ...) y key={c.id}. Cada tarjeta muestra nombre, descripción y un conteo calculado mediante productos.filter(...).length. Al pulsarla, setCategoria actualiza el filtro local.

Justificación: las dos listas representan entidades distintas. Un arreglo se transforma en tarjetas de categorías y otro en artículos de productos, manteniendo identificadores estables y expresiones entre llaves.

App.jsx / segundo arreglo de entidades

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
053  const categorias = [
054    {
055      "id": 1,
056      "nombre": "Analgésicos",
057      "descripcion": "Alivio del dolor"
058    },
059    {
060      "id": 2,
061      "nombre": "Higiene",
062      "descripcion": "Cuidado y protección personal"
063    },
064    {
065      "id": 3,
066      "nombre": "Vitaminas",
067      "descripcion": "Complementos nutricionales"
068    }
069  ]
```

E10. Arreglo de categorías declarado dentro de App.jsx.

App.jsx / categorías y claves id

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
183  <section id="categorias"><div className="section-head"><div><span
className="eyebrow">02 / ORGANIZACIÓN</span><h2>Categorías</h2></div>
<span>Selecciona para filtrar</span></div><div className="categories">
{categorias.map(c => <button key={c.id} data-testid="categoria" className=
{`category ${categoria === c.id ? 'selected' : ''}`} aria-pressed={categoria ===
c.id} aria-label={`Filtrar por ${c.nombre}`} onClick={() => setCategoria(c.id)}>
<h3>{c.nombre}</h3><p>{c.descripcion}</p><strong>{productos.filter(p =>
p.categoria === c.id).length} productos</strong></button>)}</div></section>
```

E10. Renderizado con map y key; conteo y filtro por categoría.

11. Condicionales y estilos de la investigación

La investigación usa el stock de Producto para las dos condiciones: el ternario muestra disponibilidad y && agrega el aviso de pocas unidades. La lista vacía tiene un mensaje independiente. El ejemplo de cero existencias no recibe la advertencia de pocas unidades porque se exige stock > 0.

className asigna product, price, badge, available y sold. App.css define sus colores y distribución. La barra de ventas utiliza style con un objeto JavaScript y un ancho derivado del número de detalles. Así se cumplen estilos externos y en línea dentro de la misma interfaz.

Cómo demostrarlo: señalar Ibuprofeno agotado y Alcohol con últimas unidades; escribir un nombre inexistente y pulsar Limpiar filtros para regresar al catálogo. Para demostrar específicamente el arreglo vacío, sustituir productos por [] y después restaurarlo.

App.jsx / map, key y condiciones

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
178  {visibles.length === 0 ? <p className="empty">No hay registros</p> :
visibles.map(p => <article className="product" key={p.id} data-
testid="producto"><div><small>{p.categoria_nombre}</small><h3>{p.nombre}</h3><p>
{p.descripcion}</p></div><div className="price"><strong>{moneda(p.precio)}
</strong><span className={p.stock > 0 ? 'badge available' : 'badge sold'}>
{p.stock > 0 ? `Disponible · ${p.stock}` : 'Agotado'}</span>

180  {p.stock > 0 && p.stock <= 3 && <small className="warning">¡Últimas
unidades!</small>}
```

E11. Condiciones reales en App.jsx.



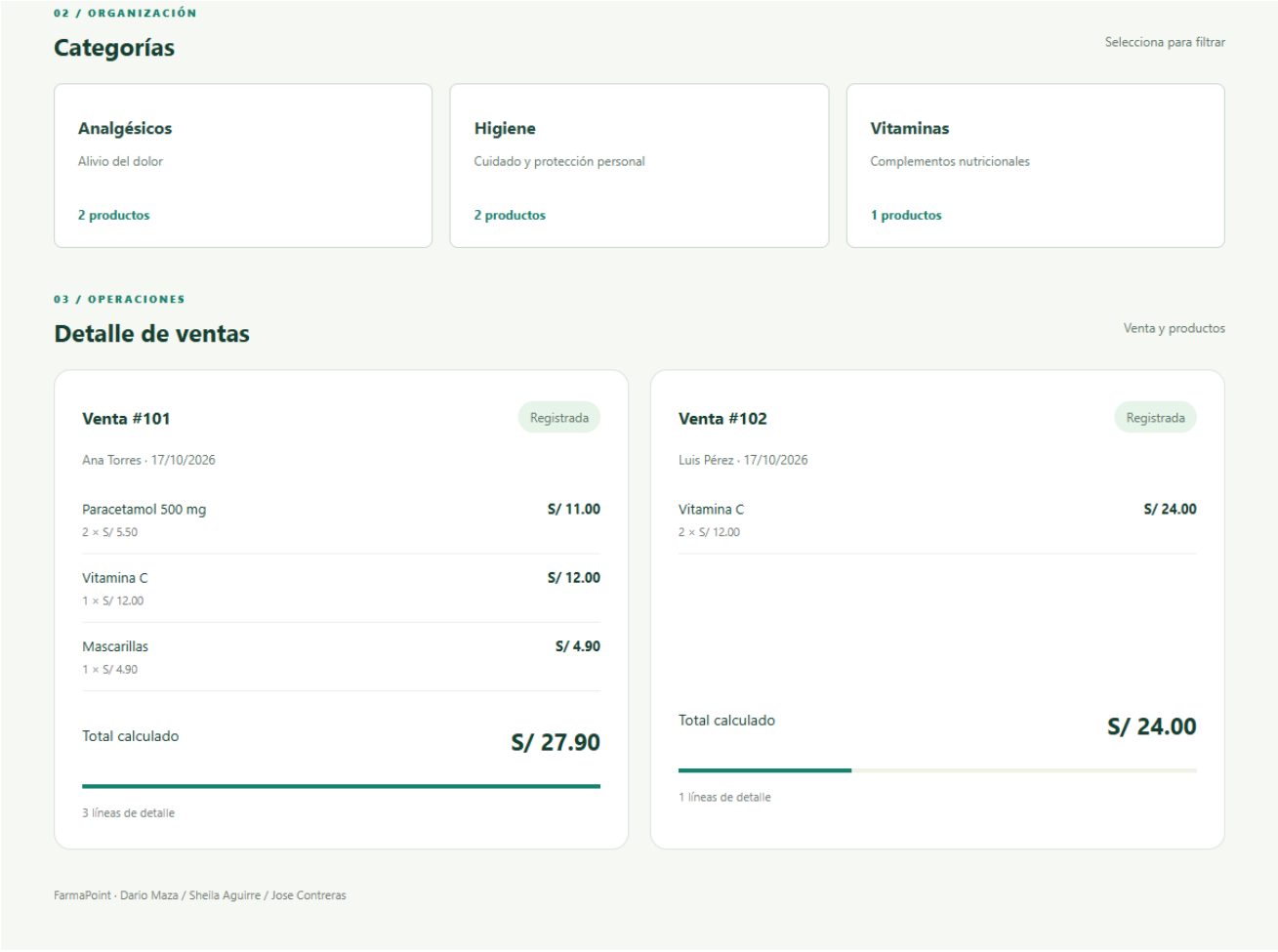
E11. Mensaje de lista vacía comprobado.

12. Modelo intermedio de la investigación

Venta.productos declara ManyToManyField con through=DetalleVenta. En React la relación se representa con ventas[].detalles[].producto. Cada venta contiene un arreglo de líneas; cada línea conserva cantidad y precio_unitario, además del producto anidado.

El map exterior identifica la venta y el interior identifica el detalle. totalVenta usa reduce para sumar los importes. También se muestra v.detalles.length: tres líneas en la venta 101 y una en la 102. No se usa el id del producto como clave del detalle; la entidad renderizada tiene su propio id.

Cómo demostrarlo: abrir Detalle de ventas, calcular manualmente 11.00 + 12.00 + 4.90 y comparar con S/ 27.90. Después localizar ventas.map y v.detalles.map en App.jsx. El campo total del JSON se conserva como referencia, pero la pantalla muestra el total calculado.



E12. Relación del proyecto de investigación, total y conteo por venta.

13. Verificación final

npm run lint y npm run build finalizaron con código 0. La plantilla instalada usa **Oxlint** en su script lint. Vite compiló 17 módulos. Las pruebas de Edge registraron cero advertencias y cero errores de página, además de cero solicitudes a la API.

Se comprobó que git ls-files frontend/node_modules frontend/dist no devuelve archivos. Ambas carpetas están ignoradas. package-lock.json sí se publica porque fija dependencias. El diagnóstico refleja las acciones probadas, no una garantía sobre escenarios futuros.

Comandos y registro de consola del navegador

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
$ npm run lint

> frontend@0.0.0 lint
> oxlint


Exit code: 0
$ npm run build

> frontend@0.0.0 build
> vite build --configLoader native

vite v8.3.4 building client environment for production...
transforming...
✓ 17 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.70 kB │ gzip:  0.44 kB
dist/assets/index-BmG1Cu3y.css    4.38 kB │ gzip:  1.52 kB
dist/assets/index-BPKoS1_j.js   226.74 kB │ gzip: 70.64 kB

✓ built in 216ms

Exit code: 0
Eventos capturados en Edge durante las pruebas:
{
  "modo": "final",
  "http": 200,
  "advertencias": [],
  "errores": [],
  "solicitudes_api": []
}

$ git ls-files frontend/node_modules frontend/dist
(sin resultados)
```

E13. Salidas reales de npm y eventos capturados del navegador. El arreglo vacío de advertencias confirma que no se observaron avisos de key.

14. Publicar en GitHub

Se actualizó el README de la raíz y el del frontend con instalación, ejecución, funciones visibles y demostración. Desde la raíz se entra a frontend, se ejecuta npm install y después npm run dev. npm run build genera los archivos locales de producción.

Repositorio: https://github.com/SAguirreT/django_project

Código: github.com/SAguirreT/django_project/tree/main/frontend

El commit 6a3a665 incorpora la interfaz y sus instrucciones. La salida de push confirma la actualización de main. Este informe y las evidencias se publican en un commit posterior para conservar la trazabilidad del código ya probado.

Git / commit y push del frontend

FarmaPoint · Lab 09 · Visor de archivos y registros de ejecución

```
$ git commit -m feat(lab09): add React inventory and nested sales interface
[main 6a3a665] feat(lab09): add React inventory and nested sales interface
 13 files changed, 2042 insertions(+), 1 deletion(-)
 create mode 100644 frontend/.gitignore
 create mode 100644 frontend/.oxlintrc.json
 create mode 100644 frontend/README.md
 create mode 100644 frontend/index.html
 create mode 100644 frontend/package-lock.json
 create mode 100644 frontend/package.json
 create mode 100644 frontend/src/App.css
 create mode 100644 frontend/src/App.jsx
 create mode 100644 frontend/src/index.css
 create mode 100644 frontend/src/main.jsx
 create mode 100644 frontend/vite.config.js

$ git push origin main
To https://github.com/SAguirreT/django_project.git
   6c482cc..6a3a665  main -> main

$ git rev-parse HEAD
6a3a66512b4a56ed9eda498ce7af54b3e3598742

$ git ls-files frontend/node_modules frontend/dist
```

E14. Registro del commit y publicación del frontend; no se añadieron dependencias ni dist.

Conclusiones y preparación para sustentar

1. React permite construir la vista mediante componentes, expresiones, map y condiciones. Django continúa siendo responsable del modelo de datos y de la API.

2. Conservar la forma de los serializers facilita la integración futura. En este laboratorio los importes se mantienen como cadenas y solo se convierten al calcular o formatear.

3. Las claves estables, la lista vacía y las condiciones de stock permiten mostrar cambios sin confundir registros. El modelo DetalleVenta conserva la información propia de cada venta.

4. La compilación y las pruebas del navegador comprobaron la interfaz entregada. Los archivos generados y las dependencias se excluyeron del repositorio.

Pregunta probable	Respuesta y demostración
¿Qué hace map?	Convierte cada registro en JSX. Mostrar productos y categorias en App.jsx.
¿Para qué sirve key?	Identifica elementos entre renderizados. Señalar key={p.id} y key={d.id}.
¿Qué cambia con el ternario y &&?	El ternario elige dos salidas; && añade una salida opcional. Mostrar agotado y pocas unidades.
¿Dónde está la relación?	Venta -> DetalleVenta -> Producto. Mostrar el map anidado y el total 27.90.
¿Ya consume Django?	No. Los datos son arreglos locales. Explicar fetch, estado, carga y errores como integración futura.
¿Cómo probar la lista vacía?	Vaciar temporalmente productos, guardar y ver No hay registros. Restaurar el arreglo.

Fuentes: guía GLAB-S09-YBESTARD-2026-02 proporcionada por el usuario; models.py y serializers.py del repositorio; Vite: Getting Started; React: Rendering Lists. Los registros de ejecución y capturas se conservan en lab09/evidencias/.